

When PASS Is Not Promotion

Binding evaluations to the exact input consumed

Xián Blanco Méndez

RESEARCH NOTE 001 · 14 SEPTEMBER 2026 · VERSION 1.0.1

Author's note

A convincing account can make a gap feel smaller than it is. This matters to me in writing, and it has become a practical question in this project. In this case, we could check a package and still could not establish that a particular evaluation had used it. This note follows that question as far as the available evidence allows.

Abstract

A passing check is evidence about a bounded execution. It does not, by itself, establish which candidate may be promoted or treated as authoritative state. This note draws on a private engineering review in which package integrity was verifiable but the exact input of a historical evaluation could not be uniquely established from the recovered records. A separate, synthetic counterexample shows why TEST_PASS cannot fill that gap. The required property is a trustworthy relationship between the execution, the material input actually consumed and the resulting evidence. Existing provenance work already addresses closely related relationships. The contribution here is an applied failure analysis and a narrow decision rule: preserve unresolved attribution instead of converting a plausible package into an evaluated candidate.

1. The observed gap

The retained records described two distinct emissions of an evidence package, both with reported integrity checks marked PASS. One package was available for direct inspection. Its archive and manifest contents were rechecked successfully while preparing this note. The other emission was represented by recorded identities; its exact contents were unavailable in the reviewed recovery evidence.

The recovery record did not establish a unique link from the historical evaluation to either emission. The recorded decision was to withhold resumption. A subsequent comparison also left input identity unresolved and the substantive evaluation not evaluated. These are historical findings at the reviewed evidence cut, not a claim about the project's current deployment state.

The observed PASS concerned package integrity. This note does not claim that the unidentified historical evaluation passed. Nor does it claim that both packages were semantically valid or equivalent.

The missing relationship was specific: which exact input did that execution consume? Package availability, an intact manifest and a recorded successful delivery could not answer it. A later successful check would establish new evidence; it would not determine the earlier execution's input.

Evidence boundary

This is a sanitized case account derived from retained machine records and documentary recovery artifacts. Private records are not included, so a public reader cannot independently verify the historical incident from this release. The synthetic example on page 2 is inspectable and reproducible. It illustrates the inference failure; it is not a reconstruction of the private execution.

2. A minimal counterexample

Consider two different input snapshots, A and B. A test checks one limited property: whether a required field is present and has the expected type. Both snapshots satisfy that property. The surviving test receipt contains only a run label and PASS. A separate inventory lists both input digests, without connecting either digest to that test run.

Possible history	Input actually tested	Retained test receipt
World A	Snapshot A	run-7, TEST_PASS
World B	Snapshot B	run-7, TEST_PASS

Both histories produce the same retained receipt. Even if someone later proves A and B each have intact bytes, the record still cannot distinguish which history occurred. Selecting B because it is available or newer adds an assumption; it does not recover the missing relationship.

Integrity of A + integrity of B + TEST_PASS ≠ evidence that run-7 tested B.

The companion [binding_example.py](#) constructs these two worlds and checks that their unbound receipts are equal. It then demonstrates a bounded matching rule with receipts that include a run identity and the digest of the bytes evaluated. Missing evidence yields UNKNOWN; a contradictory run or digest yields REJECTED; a matching pair yields BINDING_CHECK_PASS_ONLY.

The example trusts its local recorder and hashes the same immutable byte value that its evaluator parses. It does not implement secure attestation, durable storage, authority verification or production admission. Its positive result is deliberately narrower than PROMOTABLE.

Three different claims

Label used in this note	Meaning
TEST_PASS	A specified test reported success, within its coverage and execution conditions.
PROMOTABLE	The candidate satisfies the applicable admission policy, including required evidence and authorization.
CANONICAL	The candidate has actually been established as authoritative state through the required transition.

These are local definitions, not universal terms from a standard. A passing test may be one admission requirement. Satisfying admission requirements is still distinct from applying the transition and proving that the intended consumer observes the resulting state.

3. The property to require

For an admission policy that depends on evaluation of an exact candidate, evidence should support a checkable chain:

Identified execution → material input consumed → evaluation result → candidate presented for admission.

At minimum, the evidence must disambiguate the run, identify its material inputs and bind the result to the same subject. A useful implementation may preserve a receipt before consumption and retain the same binding in terminal evidence. A signed manifest alone is insufficient if the evaluator can consume different bytes.

The identity scope must be explicit. Hashing a delivery archive identifies that archive; it does not automatically cover external files, resolved configuration or later responses read during execution. For transformed inputs, preserve the relationship from source bytes to the effective representation evaluated. For live inputs, capture the observations within the declared boundary. No single package digest proves completeness of an undeclared input set.

Prevent substitution between measurement and consumption through mechanisms appropriate to the environment, such as immutable snapshots or evaluating the same captured bytes. Authenticate the recorder and verify its claims under an explicit trust model. A timestamp records a claimed time; a signature attributes a claim. Neither alone proves actual consumption or the order of events.

Falsification cases

Challenge	Required result under the stated policy
Remove the run-to-input relationship	Attribution remains UNKNOWN; withhold admission based on that evaluation.
Present the receipt for a different run or input	Reject the mismatch; preserve the original record.
Hash A, then let the evaluator read B	The observation boundary must detect or prevent substitution.
Present matching evidence with revoked authority	Input binding may pass; authorization must be assessed separately.

These are proposed acceptance conditions, not claims that the private system passed them. A complete fix would require separately evaluated evidence under its actual execution and threat model.

Recovery without rewriting history

An authenticated historical record that uniquely binds the run and consumed input could resolve the attribution gap. Proof of semantic equivalence might support a different, explicitly authorized decision, but would not identify the historical bytes consumed. If recovery remains inconclusive, retain UNKNOWN. A fresh execution with correct evidence can support a new decision while leaving the old outcome unresolved.

This approach can fit existing provenance infrastructure. It does not require a new service, ledger or architecture by default.

4. Prior art and contribution

A basic check of primary sources was completed on 14 September 2026. It is not a systematic review or a novelty determination.

Prior work	Relationship to this note
W3C PROV-DM, 2013 [1]	Models entities, activities, usage and generation. The execution–input relationship is established provenance territory; a model alone does not guarantee faithful capture.
in-toto, 2019; specification 1.0 [2]	Records step materials and products and verifies relationships against an authorized layout. It directly addresses linking artifacts across supply-chain steps.
SLSA v1.2 [3]	Describes build provenance and checking artifact identity, trusted builders and expected parameters. Its guarantees depend on scope and trust assumptions.
MITRE CWE-367 [4]	Describes changes between checking and using a resource. This motivates substitution checks, but a race was not established as the cause of the private finding.

Contribution: a concrete case of unresolved attribution despite passing integrity checks, a small counterexample, and an explicit separation of testing, admission eligibility and authoritative state. This note does not introduce provenance or claim an advance over in-toto or SLSA.

Limits and disclaimer

This is an engineering research note, not a peer-reviewed paper, certification or security audit. It makes no claim of production readiness, complete input capture, economic performance or a solved general problem of autonomous-system governance. The private case is author-reported to public readers. The example is synthetic and assumes an honest local recorder. No independent adjudication is claimed.

The conclusion is conditional: where policy requires evidence that an exact candidate was evaluated, TEST_PASS without adequate attribution cannot establish that requirement. Even adequate attribution does not itself create promotion authority or apply a canonical transition.

References

[1] Moreau, L. and Missier, P., eds. PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013, sections 2.1 and 5.1. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>

[2] Torres-Arias, S. et al. in-toto: Providing farm-to-table guarantees for bits and bytes. USENIX Security 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias> · Specification 1.0.0, sections 4.3.3 and 4.4: <https://github.com/in-toto/specification/blob/master/in-toto-spec.md>

[3] SLSA v1.2. Build: Provenance; Build: Verifying artifacts. <https://slsa.dev/spec/v1.2/build-provenance> · <https://slsa.dev/spec/v1.2/verifying-artifacts>

[4] MITRE. CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition. <https://cwe.mitre.org/data/definitions/367.html>